



TITLE: Mushroom Cultivation Monitoring System

PROPONENTS:

Alemania, Chynna
Basconcillo, Gwen
Dinglasan, Allen
Icalla, Kate
Gacilo, Gybrielle
Fesalbon, Cloyd
Manibo, Harvey
Mendoza, Maria Hanah
Rabulan, Joshua

RATIONALE:

A Mushroom Cultivation Monitoring System designed to sense humidity and temperature offers essential advantages for mushroom cultivators of all scales. Firstly, it ensures precise control of environmental conditions crucial for mushroom growth. Mushrooms are highly sensitive to fluctuations in humidity and temperature, so continuous monitoring allows cultivators to maintain optimal conditions, promoting healthy growth and maximizing yields.

Secondly, by providing real-time data on humidity and temperature levels, the monitoring system enables cultivators to track environmental parameters accurately. This data allows for prompt identification of any deviations from the ideal conditions, empowering cultivators to take immediate corrective actions. For instance, if humidity levels drop below the desired range, the system can trigger automated humidification systems to restore optimal conditions promptly.

Furthermore, automating the monitoring process enhances operational efficiency and reduces labor requirements. Rather than relying on manual checks, which are time-consuming and prone to errors, cultivators can depend on automated sensors to provide accurate data continuously. This frees up valuable time and resources while ensuring consistent cultivation practices.

Additionally, remote access capabilities allow cultivators to monitor humidity and temperature levels from anywhere, facilitating proactive interventions and adjustments. Whether overseeing a single cultivation facility or managing multiple sites, remote monitoring ensures timely responses to environmental changes, optimizing crop quality and productivity.

In conclusion, a Mushroom Cultivation Monitoring System equipped with humidity and temperature sensors is indispensable for cultivators seeking to optimize their operations. By enabling precise control, real-time monitoring, and remote access, the system empowers cultivators to achieve better results while reducing manual effort and operational risks.

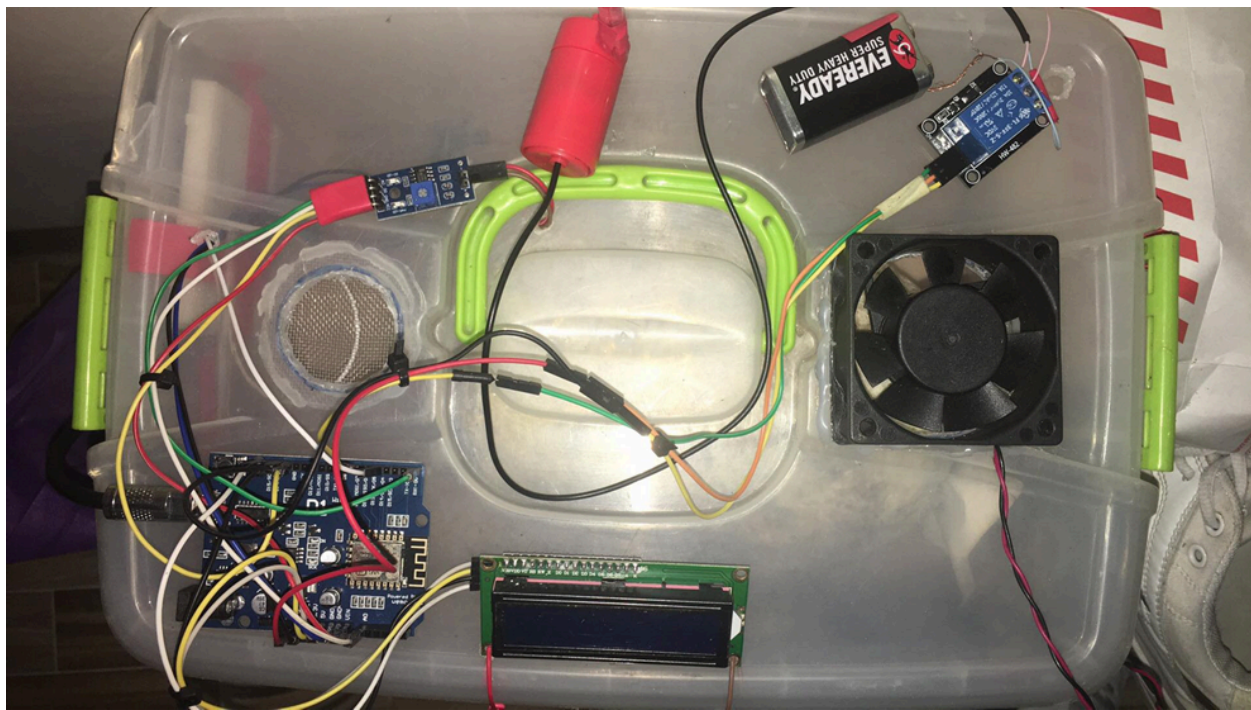


BUDGETARY PROPOSAL:

NO	ITEM	QUANTITY	UNIT PRICE	TOTAL PRICE
1	Temperature Sensor	1	92.34	
2	Soil Moisture Sensor	1	54.00	
3	Humidity Sensor	1	69.00	
4	5v Water pump with Relay Module	1	69.00	
5	Cooling Fan	1	72.00	
6	Wemos D1 R1	1	138.00	
7	LCD Display	1	95.00	
8	Jumping Wires	14	49.00	
9	Container	1	150.00	

TOTAL PRICE: 788.34

WIRINGS (DIAGRAM): (Actual Picture or clear picture of wirings including PINS or you can use Schematic Diagram)



ARDUINO SOURCE CODE:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_Sensor.h>
#include <DHT.h>
#include <DHT_U.h>
#include <MySQL_Connection.h>
#include <MySQL_Cursor.h>
#include <ESP8266WiFi.h>
```



```
// WiFi credentials
const char* ssid = "hidden network";    // Replace with your Wi-Fi SSID
const char* password = "B5BA19E3"; // Replace with your Wi-Fi password

// MySQL server connection details
IPAddress server_addr(192, 168, 254, 119); // Replace with your server IP
char db_user[] = "kate";                // Replace with your database username
char db_password[] = "kate";            // Replace with your database password

// Wemos D1 R1 pin connected to the DHT11's data pin.
#define DHT_SENSOR_PIN D4 // GPIO2 is labeled as D4 on the Wemos D1 R1

// Analog pin for the soil moisture sensor.
#define SOIL_MOISTURE_PIN A0

// Type of DHT sensor.
#define DHT_SENSOR_TYPE DHT11

// Relay pin
const int relayPin = D7; // Adjust the pin number if necessary

// Initialize DHT sensor.
DHT dht_sensor(DHT_SENSOR_PIN, DHT_SENSOR_TYPE);

// Initialize the LCD, set the I2C address to 0x27 for a 16 chars and 2 line display
LiquidCrystal_I2C lcd(0x27, 16, 2);

// MySQL client objects
WiFiClient client;
MySQL_Connection conn(&client);
MySQL_Cursor* cursor;

void setup() {
    // Initialize serial communication at a baud rate of 115200
    Serial.begin(9600);
    Serial.println("DHT11 and Soil Moisture sensor test!");

    // Initialize the DHT sensor
    dht_sensor.begin();

    // Initialize the I2C communication on pins D14 (SDA) and D15 (SCL)
    Wire.begin(D14, D15);

    // Initialize the LCD
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
    lcd.print("Humidity:");
    lcd.setCursor(0, 1);
    lcd.print("Temp:");
}
```



```
// Connect to Wi-Fi
WiFi.begin(ssid, password);
Serial.print("Connecting to Wi-Fi");
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nWiFi connected");

// Connect to MySQL server
if (conn.connect(server_addr, 3306, db_user, db_password)) {
    Serial.println("Connected to MySQL server");
} else {
    Serial.println("Connection failed.");
    return;
}

cursor = new MySQL_Cursor(&conn);

// Initialize the relay pin
pinMode(relayPin, OUTPUT);
digitalWrite(relayPin, LOW); // Ensure relay is off initially
}

void loop() {
    // Read humidity
    float humi = dht_sensor.readHumidity();
    // Read temperature in Celsius
    float temperature_C = dht_sensor.readTemperature();
    // Read temperature in Fahrenheit
    float temperature_F = dht_sensor.readTemperature(true);

    // Read soil moisture
    int soil_moisture = analogRead(SOIL_MOISTURE_PIN);

    // Convert soil moisture value to percentage
    int soilMoisturePercent = map(soil_moisture, 1023, 0, 0, 100);
    soilMoisturePercent = constrain(soilMoisturePercent, 0, 100);

    // Check whether the DHT sensor reading is successful or not
    if (isnan(temperature_C) || isnan(temperature_F) || isnan(humi)) {
        Serial.println("Failed to read from DHT sensor!");
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("DHT Sensor error!");
    } else {
        // Display DHT sensor data on Serial Monitor
        Serial.print("Humidity: ");
        Serial.print(humi);
        Serial.print("% | ");
        Serial.print("Temperature: ");
        Serial.print(temperature_C);
```



```
Serial.print("°C ~ ");
Serial.print(temperature_F);
Serial.println("°F");

// Display DHT sensor data on LCD
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("H:");
lcd.print(humi);
lcd.print("%");

lcd.setCursor(0, 1);
lcd.print("T:");
lcd.print(temperature_C);
lcd.print("C/");
lcd.print(temperature_F);
lcd.print("F");

// Display soil moisture data on Serial Monitor
Serial.print("Soil Moisture: ");
Serial.print(soil_moisture);
Serial.print(" (");
Serial.print(soilMoisturePercent);
Serial.println("%)");

// Display soil moisture data on LCD
lcd.setCursor(10, 0); // Adjust cursor position for soil moisture
lcd.print("S:");
lcd.print(soilMoisturePercent);
lcd.print("%");

// Control the relay based on soil moisture percentage
bool pumpStatus = false;
if (soilMoisturePercent < 40) {
    digitalWrite(relayPin, HIGH); // Turn relay on
    pumpStatus = true;
    Serial.println("Soil is dry. Turning on the water pump.");
} else {
    digitalWrite(relayPin, LOW); // Turn relay off
    Serial.println("Soil moisture is adequate. Turning off the water pump.");
}

// Display water pump status on LCD
lcd.setCursor(10, 1); // Adjust cursor position for water pump status
lcd.print("P:");
if (pumpStatus) {
    lcd.print("ON");
} else {
    lcd.print("OFF");
}

// Send data to the MySQL database
```



```
char query[256];
    sprintf(query, "INSERT INTO sensors_data.sensors_readings (humidity, temperature_c, temperature_f,
soil_moisture, water_pump_status) VALUES (%.2f, %.2f, %.2f, %d, %d)", humi, temperature_C,
temperature_F, soilMoisturePercent, pumpStatus);
    cursor->execute(query);
}

// Wait 2 seconds between readings
delay(2000);
}
```